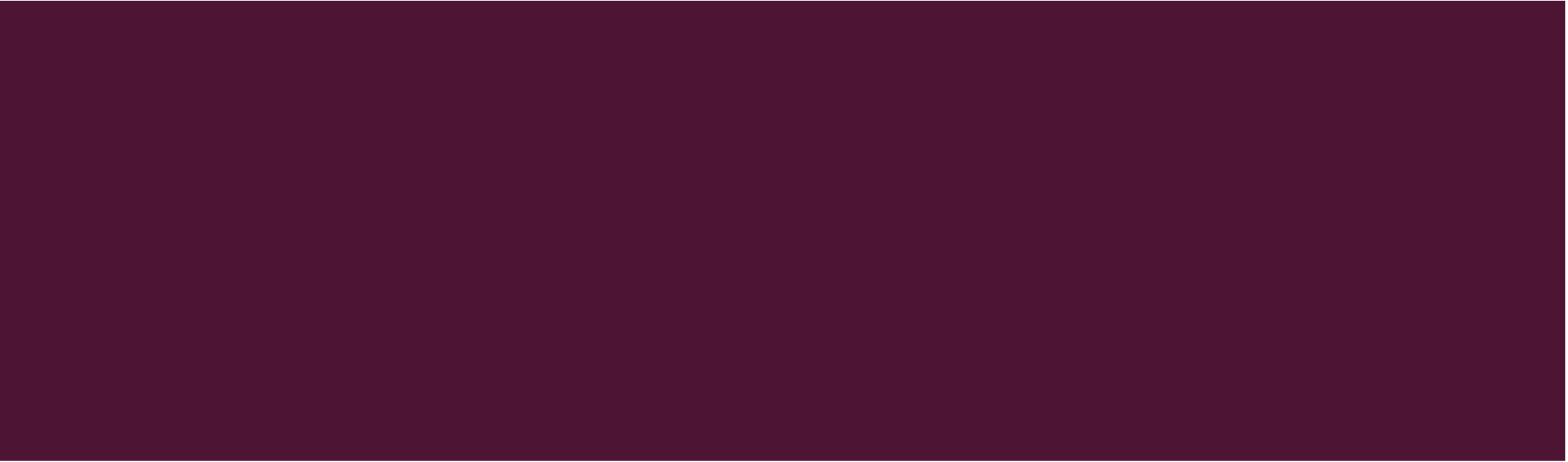

CHAPTER 5: PUBLIC KEY CRYPTOGRAPHY



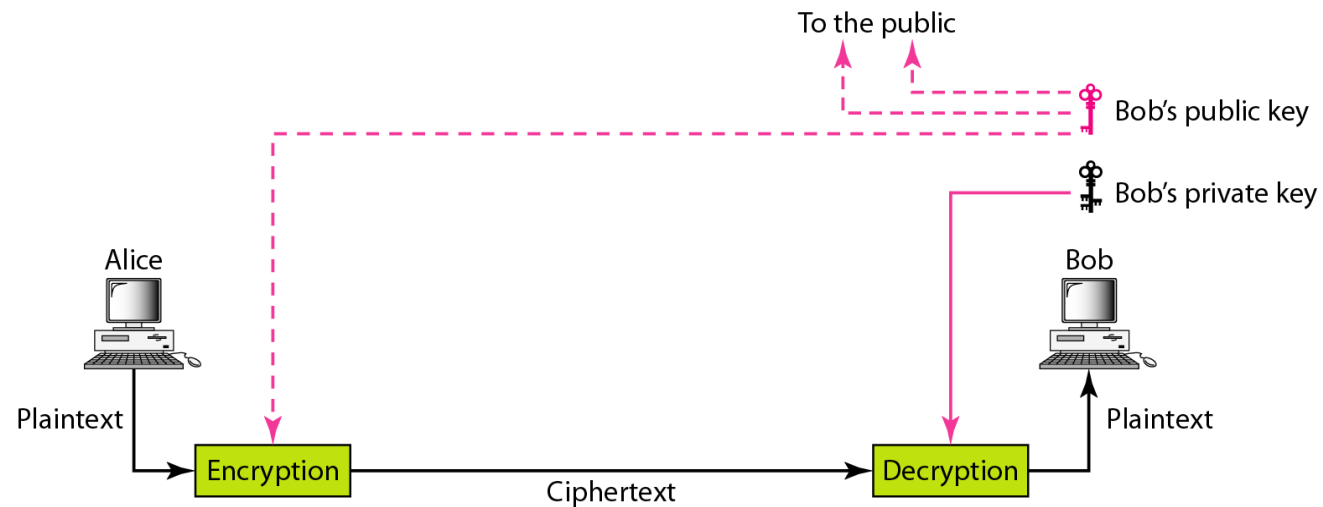
PUBLIC KEY CRYPTOGRAPHY

- **The issue with secret-key encryption**
 - Both the sender and the receiver must first agree on the key to be used for encryption (which may also involve determining the method of key exchange—online or offline).
 - Both parties must utilize a secure communication channel—one that prevents eavesdroppers from intercepting the key—to exchange that key.
 - However, in reality, communication channels are inherently insecure!
- Consequently, techniques were subsequently developed to enable the **secure distribution of keys** over networks; these methods employ distinct keys for encryption and decryption. This approach is known as **Public Key Cryptography**, or alternatively, **Asymmetric Key Encryption**.

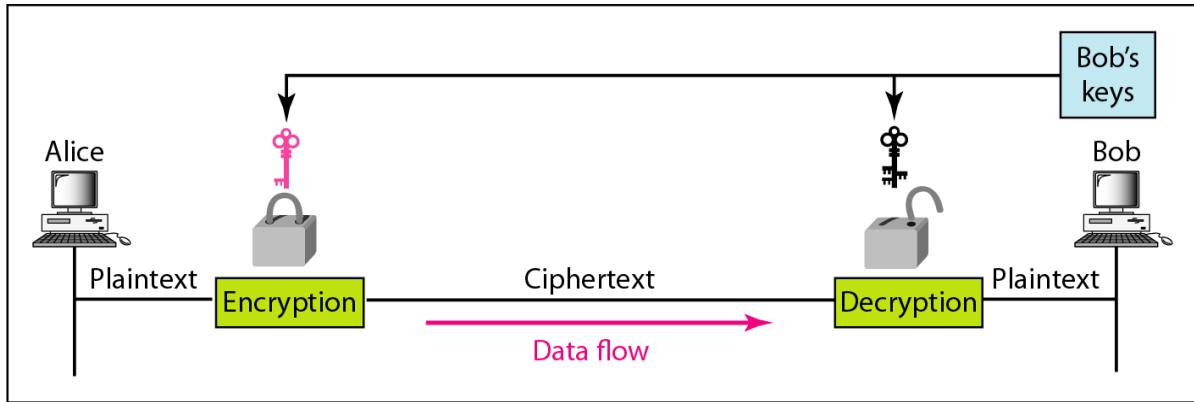
PUBLIC KEY CRYPTOGRAPHY (CONT.)

This encryption method utilizes two keys:

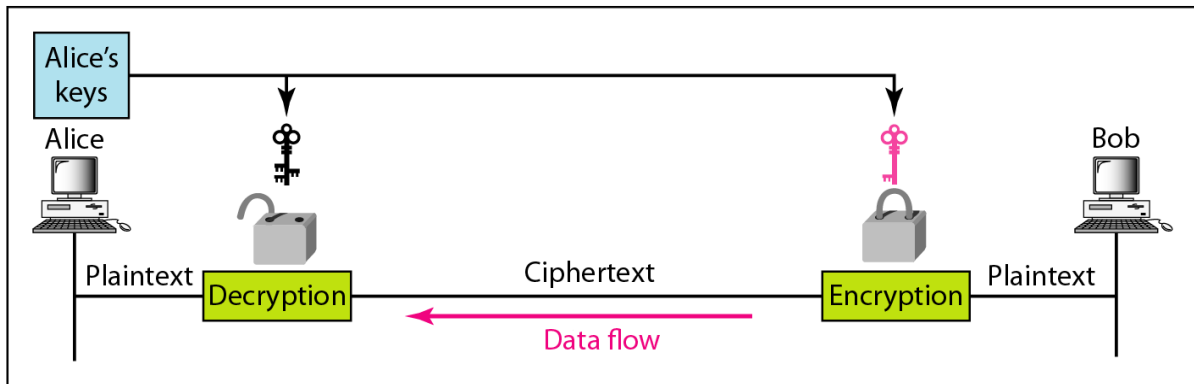
- A **Public Key**, which is made *known to everyone* to identify the sender of the data.
- A **Private Key**, which must be *kept secret* and known only to the owner, enabling them to decrypt data that was encrypted using the Public Key.



PUBLIC KEY CRYPTOGRAPHY (CONT.)



a. Bob's keys are used in Alice-Bob communication

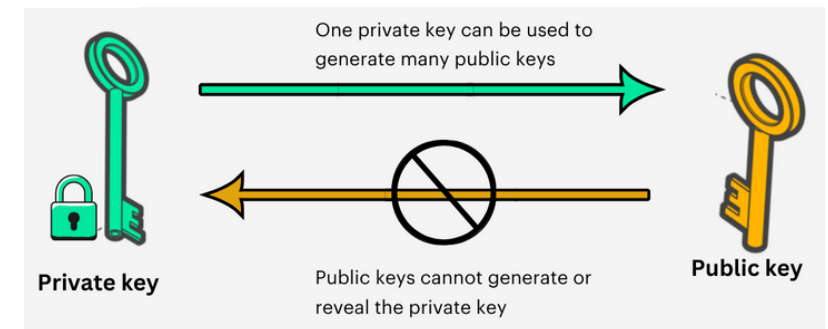


b. Alice's keys are used in Bob-Alice communication

- This type of encryption safeguards the **confidentiality** and **integrity** of data transmitted from the sender to the recipient, as only the intended recipient possesses the private key required to decrypt the data.

PUBLIC KEY CRYPTOGRAPHY (CONT.)

- Modern public-key cryptography was described in 1976 by **Professor Martin Hellman** of Stanford University and doctoral student **Whitefield Diffie**.
- They outlined a key-pair system that enables two parties to communicate securely without needing to share a single secret key.
- The process of generating a key pair relies on a concept known as a **one-way function/trapdoor function**—a mathematical function that is *easy to compute* in one direction but *extremely difficult to reverse* to find the original input.
 - **Multiplication vs. Factorization**
 - **Exponentiation vs. Logarithms**



PUBLIC KEY CRYPTOGRAPHY (CONT.)

Multiplication vs. Factorization

- Suppose we have the numbers **9** and **16**; calculating their product yields **144** immediately.
- Conversely, if we **start with the number 144** and want to determine *which two numbers multiply to produce it*, finding the answer would certainly take longer than the multiplication itself.
- There are **eight pairs** of numbers that multiply to 144: **144×1**, **72×2**, **48×3**, **36×4**, **24×6**, **18×8**, **16×9**, and **12×12**; one would then have to decide which of these pairs represents the correct answer.

PUBLIC KEY CRYPTOGRAPHY (CONT.)

Exponentiation vs. Logarithms

- Suppose you want to calculate 3 to the power of 6; this is easily done: $3^6 = 729$.
- Conversely, if you **start with the number 729** and wish to *find two numbers, x and y*, such that $\log_x(729) = y$, the process takes longer, and you must also select the correct answer from the resulting pair of numbers.

PUBLIC KEY CRYPTOGRAPHY (CONT.)

- The **one-way function concept** described above serves as an example to explain the keys used in public-key cryptography, where the **key pair** used for encryption and decryption typically shares a mathematical relationship.
- (From an eavesdropper's perspective) **Even if one key is known, it is difficult to calculate the value of the other**; one key is used for encryption and the other for decryption, and they are even interchangeable!
- Another key feature is that while *either key can be used for encryption, the other key must be used for decryption*; thus, both keys are always required for the encryption and decryption processes. For this reason, this type of encryption is also known as Asymmetric Cryptography.

PUBLIC KEY CRYPTOGRAPHY (CONT.)

- There are several widely known public-key encryption algorithms, such as...
 - RSA
 - Diffie-Hellman
 - Digital Signature
 - ElGamal
 - Elliptic Curve Cryptography (ECC)
 - Public-Key Cryptography Standard (PKCS)
 - Cramer-Shoup
 - Key Exchange Algorithm (KEA)
 - LUC
- etc.

PUBLIC KEY CRYPTOGRAPHY: RSA

- It is named after the initials of its inventors—**Ron Rivest**, **Adi Shamir**, and **Len Adleman** from MIT.
- First published in 1977, this algorithm can be used for *data encryption* as well as for *digital signatures*.
- Data encrypted with a private key can be decrypted using the corresponding public key, and vice versa.
- RSA is widely used in a variety of products, platforms, and industries.
- RSA can also be found on hardware-secured telephones, Ethernet network cards, and smart cards.



Shamir Rivest Adleman

PUBLIC KEY CRYPTOGRAPHY: RSA (CONT.)

RSA encryption involves three key steps: **Key generation, Encryption, and Decryption**

The generation of the public and private keys:

- 1) Select **prime numbers** p and q .

The guiding principle for this selection is that larger numbers—such as 1,024-bit or 2,048-bit values—make the keys much harder to guess (though this also results in slower encryption and decryption processes).

The first 50 prime numbers are

2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137, 139, 149, 151, 157, 163, 167, 173, 179, 181, 191, 193, 197, 199, 211, 223, 227 and 229

www.mathsisfun.com/prime_numbers.html

PUBLIC KEY CRYPTOGRAPHY: RSA (CONT.)

- 2) Calculate values **n** and **z**, where **$n = (p)(q)$** and **$z = (p-1)(q-1)$** .
- 3) Choose a value **e** such that **$e < n$** and *e shares no common divisor with z other than 1* ($\text{gcd}(z, e) = 1$).
 - e and z are relatively prime
- 4) Calculate the value **d** such that **$ed \bmod z = 1$** .
 - The **Public Key** is the pair **(n, e)**
 - The **Private Key** is the pair **(n, d)**

Question 1

Based on the key generation process described above, suppose Alice wants to send confidential data to Bob. **Determine Bob's public key and private key**, given that Bob selects the values **p = 5** and **q = 7**.

PUBLIC KEY CRYPTOGRAPHY: RSA (CONT.)

The encryption and decryption processes are as follows:

- 1) Suppose we wish to transmit data **m** (an unencrypted message in numeric form), where **m < n**.
 - Convert characters to numbers between **1 and 26** (or other numerical values, depending on the agreed-upon mapping—e.g., **a = 00**, **A = 26**, group the numbers into sets of four digits each).

Message												
A	B	C	D	E	F	G	H	I	J	K	L	M
1	2	3	4	5	6	7	8	9	10	11	12	13
N	O	P	Q	R	S	T	U	V	W	X	Y	Z
14	15	16	17	18	19	20	21	22	23	24	25	26



R	A	N	G	S	I	T
18	1	14	7	19	9	20

PUBLIC KEY CRYPTOGRAPHY: RSA (CONT.)

a = 00	b = 01	c = 02	d = 03
e = 04	f = 05	g = 06	h = 07
i = 08	j = 09	k = 10	l = 11
m = 12	n = 13	o = 14	p = 15
q = 16	r = 17	s = 18	t = 19
u = 20	v = 21	w = 22	x = 23
y = 24	z = 25		

A = 26	B = 27	C = 28	D = 29
E = 30	F = 31	G = 32	H = 33
I = 34	J = 35	K = 36	L = 37
M = 38	N = 39	O = 40	P = 41
Q = 42	R = 43	S = 44	T = 45
U = 46	V = 47	W = 48	X = 49
Y = 50	Z = 51		

0 = 52	1 = 53	2 = 54	3 = 55
4 = 56	5 = 57	6 = 58	7 = 59
8 = 60	9 = 61	= 62	. = 63
, = 64	; = 65	? = 66	

Plaintext: **How are you?**



H	o	w	
33	14	22	62
a	r	e	
00	17	04	62
y	o	u	?
24	14	20	66



$P_1 = 3314$ $P_2 = 2262$
 $P_3 = 0017$ $P_4 = 0462$
 $P_5 = 2414$ $P_6 = 2066$

PUBLIC KEY CRYPTOGRAPHY: RSA (CONT.)

- 2) Encryption utilizes the **public key (n, e)** with the formula $c = m^e \bmod n$, where **c** is the ciphertext.
- 3) Decryption utilizes the **private key (n, d)** with the formula $m = c^d \bmod n$.

Question 2

Based on the answer to Question 1, suppose Alice wants to send the message "**love**" to Bob; *what ciphertext would she generate?*
Also, show the *decryption process on Bob's side.*

PUBLIC KEY CRYPTOGRAPHY: RSA (CONT.)

Question 2

- Encrypt using **Public Key (n, e)**

Plaintext (PT)	m	m^e	$c = m^e \bmod n$	Ciphertext (CT)
I	12			
o	15			
v	22			
e	5			

PUBLIC KEY CRYPTOGRAPHY: RSA (CONT.)

Question 2

- Decrypt using **Private Key (n, d)**

CT	C	C^d	$c = m^e \bmod n$	PT

PUBLIC KEY CRYPTOGRAPHY: RSA (CONT.)

Question 3

- Suppose A wants to send the data value **24** to B, and B has generated the following keys: **Public Key = (143, 37)** and **Private Key = (143, 13)**.
- Demonstrate the process of encrypting and decrypting this data.

PUBLIC KEY CRYPTOGRAPHY: RSA (CONT.)

Question 3

- For encryption, the **public key (143, 37)** is used, based on the formula **$c = m^e \bmod n$**

PUBLIC KEY CRYPTOGRAPHY: RSA (CONT.)

Question 3

- For decryption, the **private key (143, 13)** is used, based on the formula $m = c^d \bmod n$

PUBLIC KEY CRYPTOGRAPHY: RSA (CONT.)

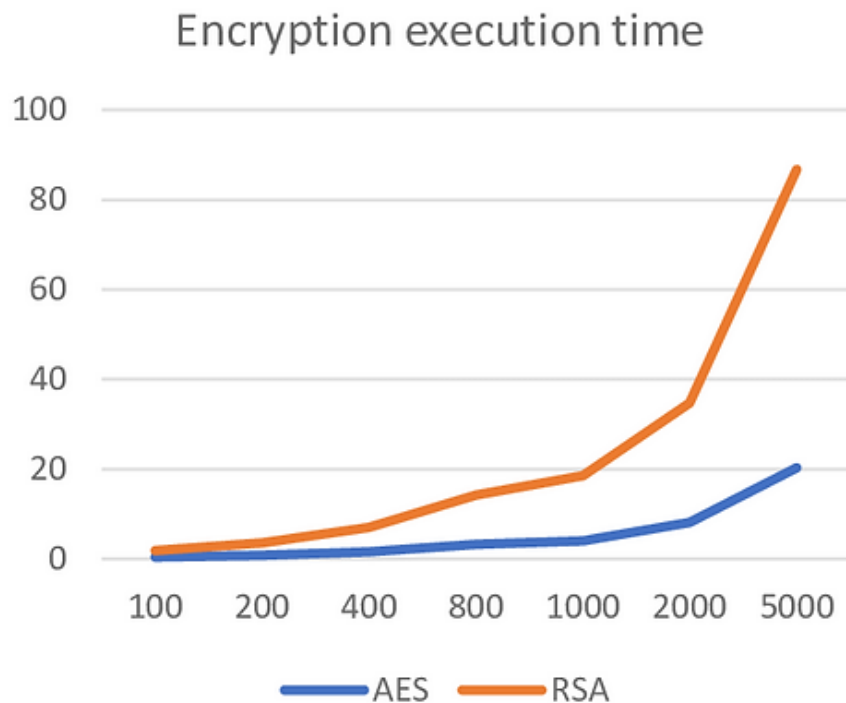
Question 4

Generate a public key and a private key to demonstrate the encryption and decryption processes for the given data as follows:

- A. $p = 3, q = 13, e = 5$ with $M = 10$
- B. $p = 7, q = 13, e = 5$ with $M = 8$
- C. $p = 11, q = 7, e = 11$ with $M = 7$
- D. $p = 5, q = 31, e = 13$ with $M = 5$

PUBLIC KEY CRYPTOGRAPHY: RSA (CONT.)

- In general, keys used on the Internet are **1024/2048 bits in size** (significantly larger than the examples shown in the slides), making the encryption and decryption processes ***much slower*** compared to symmetric-key encryption methods such as DES and RC4.



No. of words	<i>AES</i>	<i>RSA</i>
100	0.408306	1.7743
200	1.16398	3.453
400	2.33085	7.1067
800	4.6168	13.8771
1000	5.0771	18.4126
2000	10.3325	33.9662
5000	29.1165	84.3077

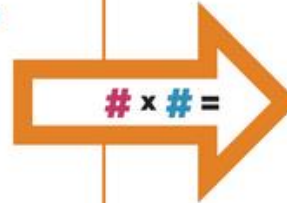
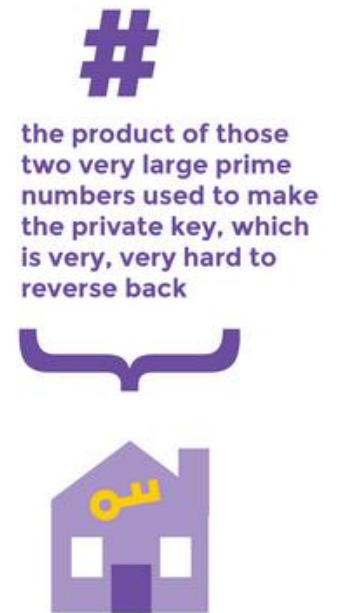
PUBLIC KEY CRYPTOGRAPHY: RSA (CONT.)

- What makes RSA difficult to crack is that even if the **public key (n, e)**, the **message (m)**, and the **ciphertext (c)** are known, it remains *difficult to reverse-calculate the private key (d)*.

PRIVATE KEY



PUBLIC KEY



PUBLIC KEY CRYPTOGRAPHY: RSA (CONT.)

**Example 1: Modulus size in the order of 1,024 bit
,composed of the two primes!!!**

p =

10933766183632575817611517034730668287155799984632223
45413874567112127345628767000829084330287552127497024
5314593222946129064538358581018615539828479146469

q =

10910616967349110231723734078614922645337060882141748
96820983422513897601117999339429981015973690446855402
1708289824396553412180514827996444845438176099727

n =

11929413484016950905552721133125564964460656966152763
80120674819549430568511503338063159570377156202973050
00118628770846689969112892212245457118060574995989517
08004210526342737632227426639311619351783957077350563
22315966811219273374739732203125125990612313222509455
06260066557538238517575390621262940383913963

Example 2: 1024-bit RSA encryption key

n =

A9E167983F39D55FF2A093415EA6798985C8355D9A915BFB1D01
DA197026170FBDA522D035856D7A986614415CCFB7B7083B09C9
91B81969376DF9651E7BD9A93324A37F3BBBAF460186363432CB
07035952FC858B3104B8CC18081448E64F1CFB5D60C4E05C1F53D
37F53D86901F105F87A70D1BE83C65F38CF1C2CAA6AA7EB

e = 010001

d =

67CD484C9A0D8F98C21B65FF22839C6DF0A6061DBCEDA7038894
F21C6B0F8B35DE0E827830CBE7BA6A56AD77C6EB517970790AA0
F4FE45E0A9B2F419DA8798D6308474E4FC596CC1C677DCA991D0
7C30A0A2C5085E217143FC0D073DF0FA6D149E4E63F01758791C
4B981C3D3DB01BDFFA253BA3C02C9805F61009D887DB0319

PUBLIC KEY CRYPTOGRAPHY (CONT.)

PUBLIC KEY GENERATOR WEB SITES

- <https://cryptotools.net/rsagen>
- <https://randomkeygen.com/rsa-key>
- <https://sshkeygenerator.com>
- <https://www.csfieldguide.org.nz/en/interactives/rsa-key-generator>
- <https://zecurit.com/tools/rsa-key-pair-generator>
- <https://www.javainuse.com/rsagenerator>
- <https://www.thehackerwire.com/hacking-tools/ssh-key-generator>
- <https://inventivehq.com/tools/security/ssh-key-generator>

RSA Key Generator

<https://cryptotools.net/rsagen>

You may generate an RSA private key with the help of this tool. Additionally, it will display the public key of a generated or pasted private key.

Key Length

1024



Generate key pair

Private key

```
-----BEGIN RSA PRIVATE KEY-----
MIICWwIBAAKBgHPkbn4eD30otsJ7mK2LK7hLRjVUDU6Lx2Ki/ucIm3PDFtIIfVaC
q0iJrDSi8WQvLaGrKckMw5VItmQpxiMt3kq9g1IkkstK9W1gjEqMMRyskBZnDoh
6OBzBrA9KDN6s14b1Vw/Jnmv8y0+8CGSTRQ3sEiQmhzM8GMGYSXhF7pPAgMBAAEC
gYA1lhV707YESpsTclBvIG7ag5dOnmb+bXkJ9YEKXvdD0SbqkZRM5W41xWjTXXFa
o65x+3g+20IspXt5+s5nBgMAF2itACvhLCzUc/3hFr0F6Xt0xN+jhYoiXk+nPLTw
EcXUu02TPmJo8rsa8lsgDq2hnNavJvy0Bt0GmtxZCwchMQJBAOVPUaZI76mhkyRH
GFKTXQ1P449TZpXNgWpyxC0TAxPpcJV0me1G8k58re44T2FDnRgagEB30iehrjj1
2AB7D8cCQQCBYarwR8aJxFRoLhzGEj00wG/XFSPLuo1pacLIrfqRJ9F3g/6NsF8m
vA590qN5cEV1mfMrm3r8eIrRT1zjMxE5AkApn5Q6ktKt/94tn5oSox0o5W9w/LqK
RptPLzXbHxzXy0/eqj3WjCI/jPIIY6NFDHjffY/4g8dp2jc9r6Jds/jRAkBCztmr
hNSXNQLk3NLM0H9A+kuGA4t9WrAz94G2QGg/ZhE6prTeEzG+/aJ6BDeXsIqdZEZ1
wQ5rqg78b/+17iu5AkeA38vZZpXz6+wwDYB1yzji9C6Bjmy/ZyTs1v9MPZReaWIP
WfcxJMEZ7UsSWMZ8fK09KJyr8lyCUMGCgUm1sUP4g==
-----END RSA PRIVATE KEY-----
```

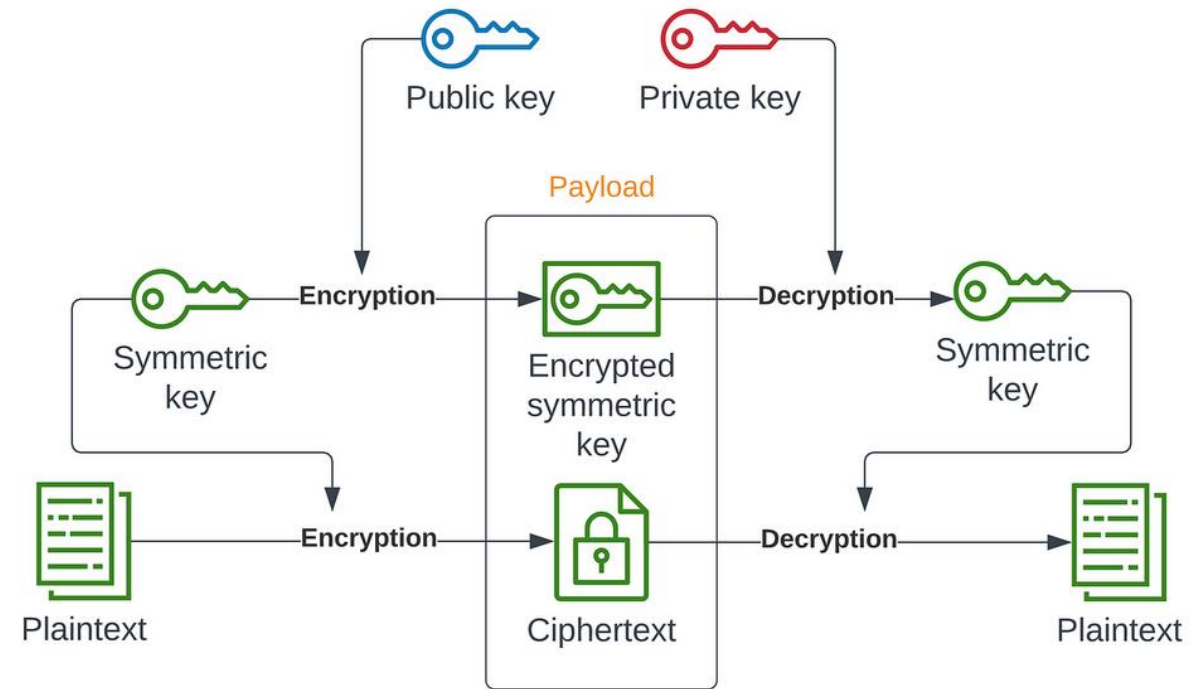
Public key

```
-----BEGIN PUBLIC KEY-----
MIGeMA0GCSqGSIb3DQEBAQUAA4GMADCBiAKBgHPkbn4eD30otsJ7mK2LK7hLRjVU
DU6Lx2Ki/ucIm3PDFtIIfVaCq0iJrDSi8WQvLaGrKckMw5VItmQpxiMt3kq9g1I
kkstK9W1gjEqMMRyskBZnDoh6OBzBrA9KDN6s14b1Vw/Jnmv8y0+8CGSTRQ3sEiQ
mhzM8GMGYSXhF7pPAgMBAAE=
-----END PUBLIC KEY-----
```

PUBLIC KEY CRYPTOGRAPHY (CONT.)

HYBRID CRYPTOGRAPHY

- In practice, RSA is often used in conjunction with AES; RSA is initially employed to *distribute the secret key* used for AES encryption—a key known as the "**session key**."
- The hybrid cryptography **combines** symmetric-key and asymmetric-key encryption techniques.
- **Asymmetric-key encryption** is used to exchange a session key—a shared key used by both the sender and the receiver to decrypt the message.
- While **symmetric-key encryption** is employed for the communication in a limited time.

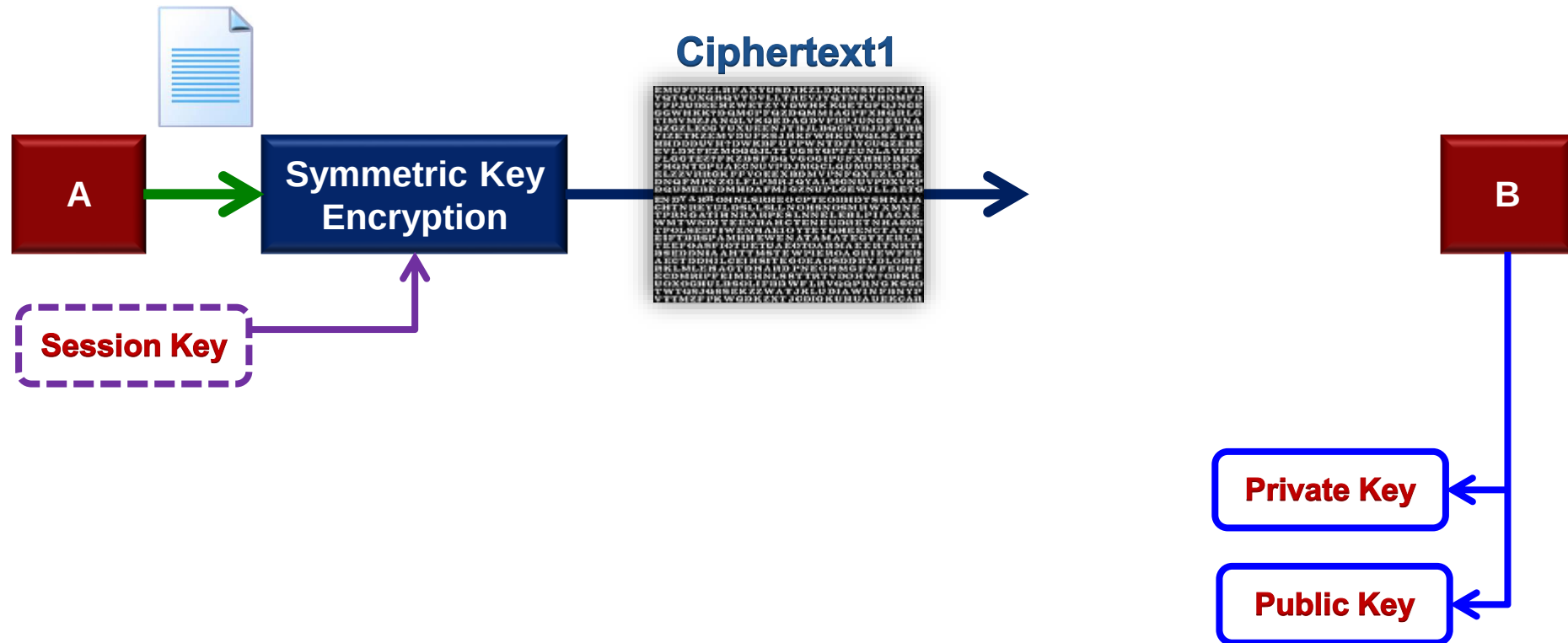


PUBLIC KEY CRYPTOGRAPHY (CONT.)

HYBRID CRYPTOGRAPHY

Suppose A wants to send data to B using hybrid encryption; the process is as follows:

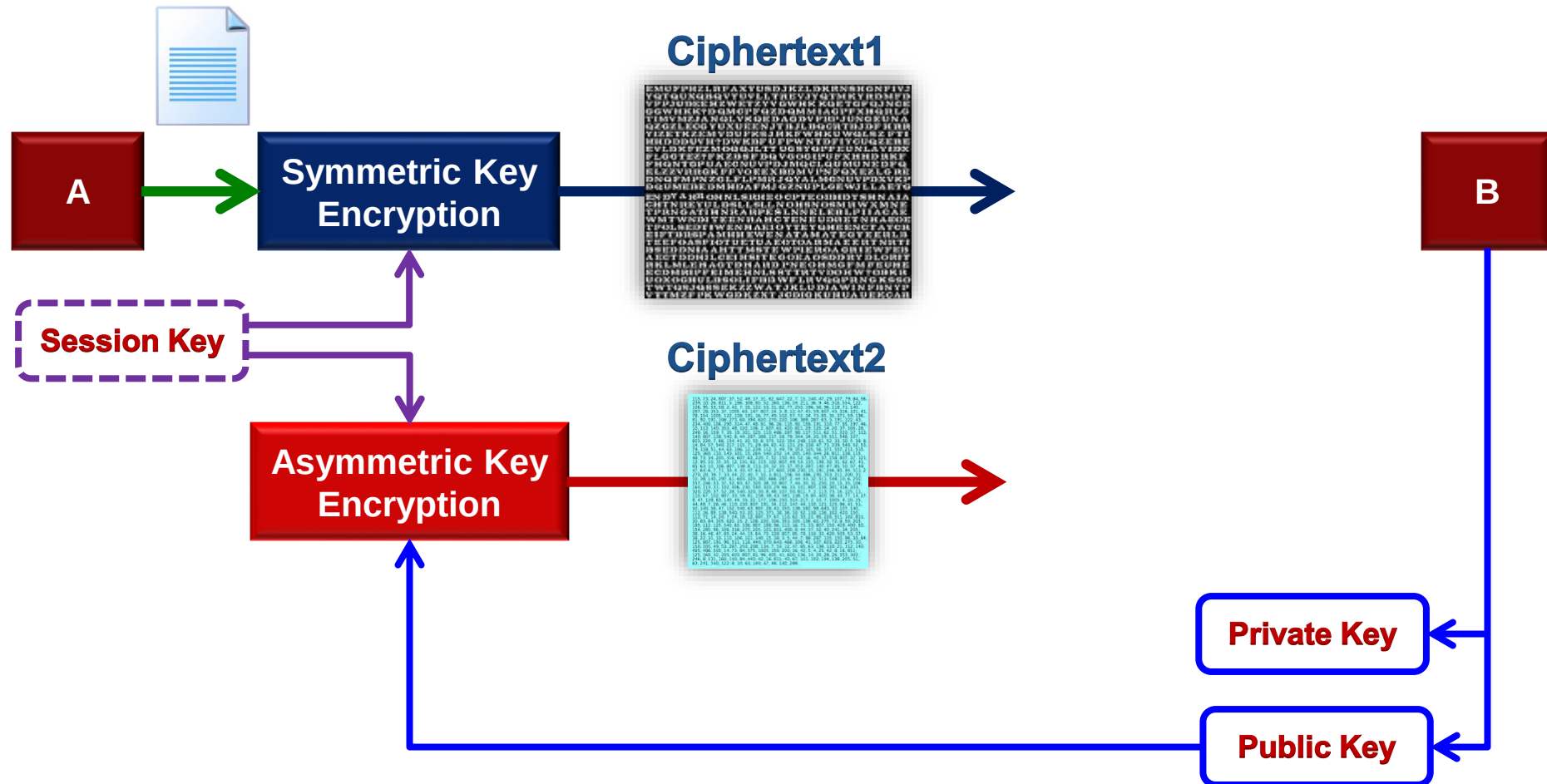
- A generates a *session key* (using a random number) and then uses this session key to encrypt the data to be sent, producing **Ciphertext1**.



PUBLIC KEY CRYPTOGRAPHY (CONT.)

HYBRID CRYPTOGRAPHY

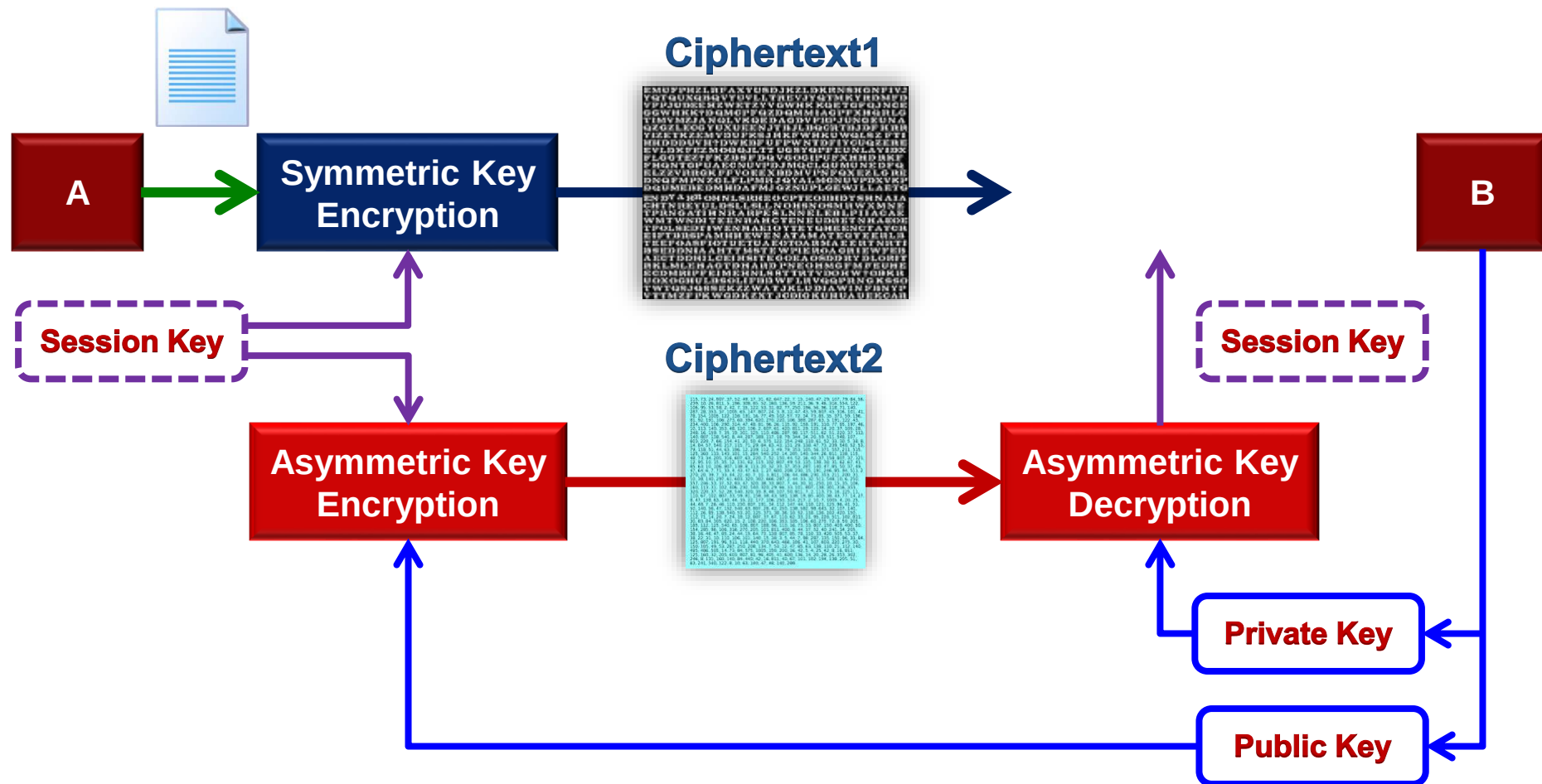
- Next, A takes B's public key to encrypt only the session key resulting in **Ciphertext2**, which is then sent to B.



PUBLIC KEY CRYPTOGRAPHY (CONT.)

HYBRID CRYPTOGRAPHY

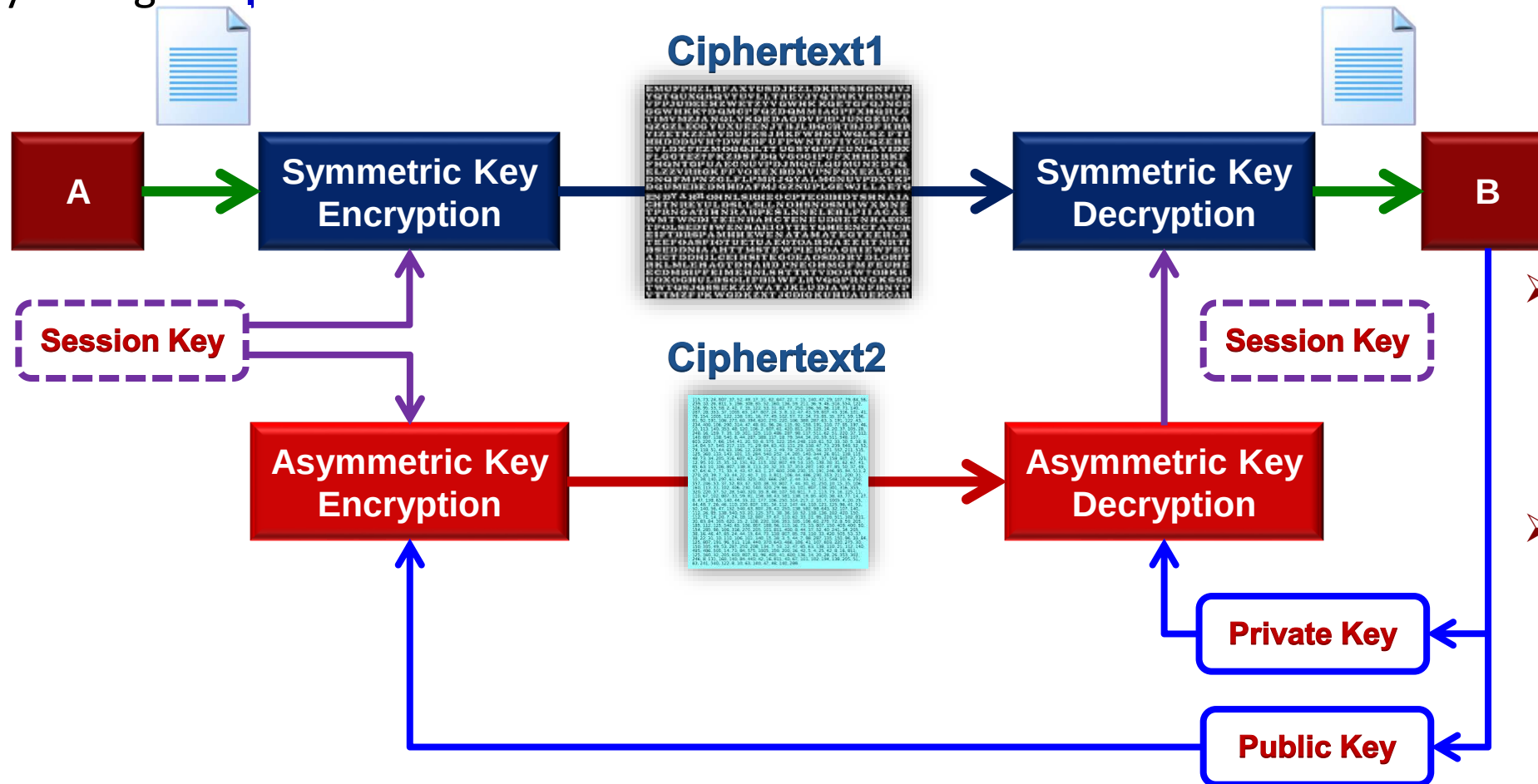
- When B receives the data, B uses their own private key to decrypt it, resulting in the **session key**.



PUBLIC KEY CRYPTOGRAPHY (CONT.)

HYBRID CRYPTOGRAPHY

- Then, B uses the session key obtained from the previous step to decrypt Ciphertext1, ultimately yielding the **plaintext**.



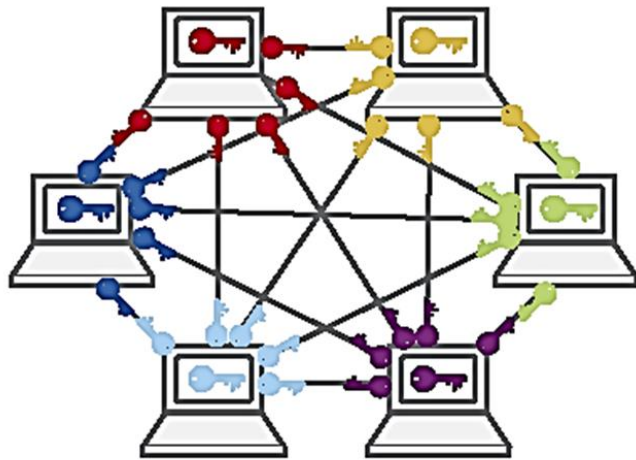
- From this point on, B can use the obtained session key to communicate with A until the key expires.
- Once the session key expires, the system will select a new session key for continued communication.

PUBLIC KEY CRYPTOGRAPHY (CONT.)

HYBRID CRYPTOGRAPHY

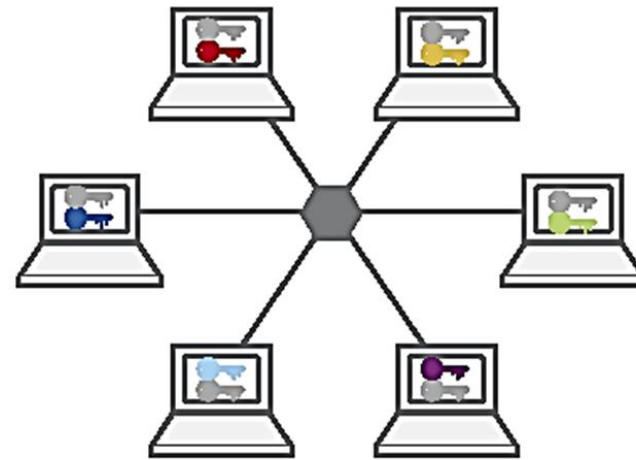
The **advantages** of this hybrid encryption method include:

- It prevents data interception or tampering during transmission; even if B's public key is intercepted, it is impossible to decrypt Ciphertext1 or Ciphertext2.
- It offers a high level of security.
- It overcomes the drawbacks of traditional key distribution methods.



SYMMETRIC

Symmetric cryptography has an equation of $\frac{n \times n - 1}{2}$ for the number of keys needed. In a situation with 1000 users, that would mean 499,500 keys.

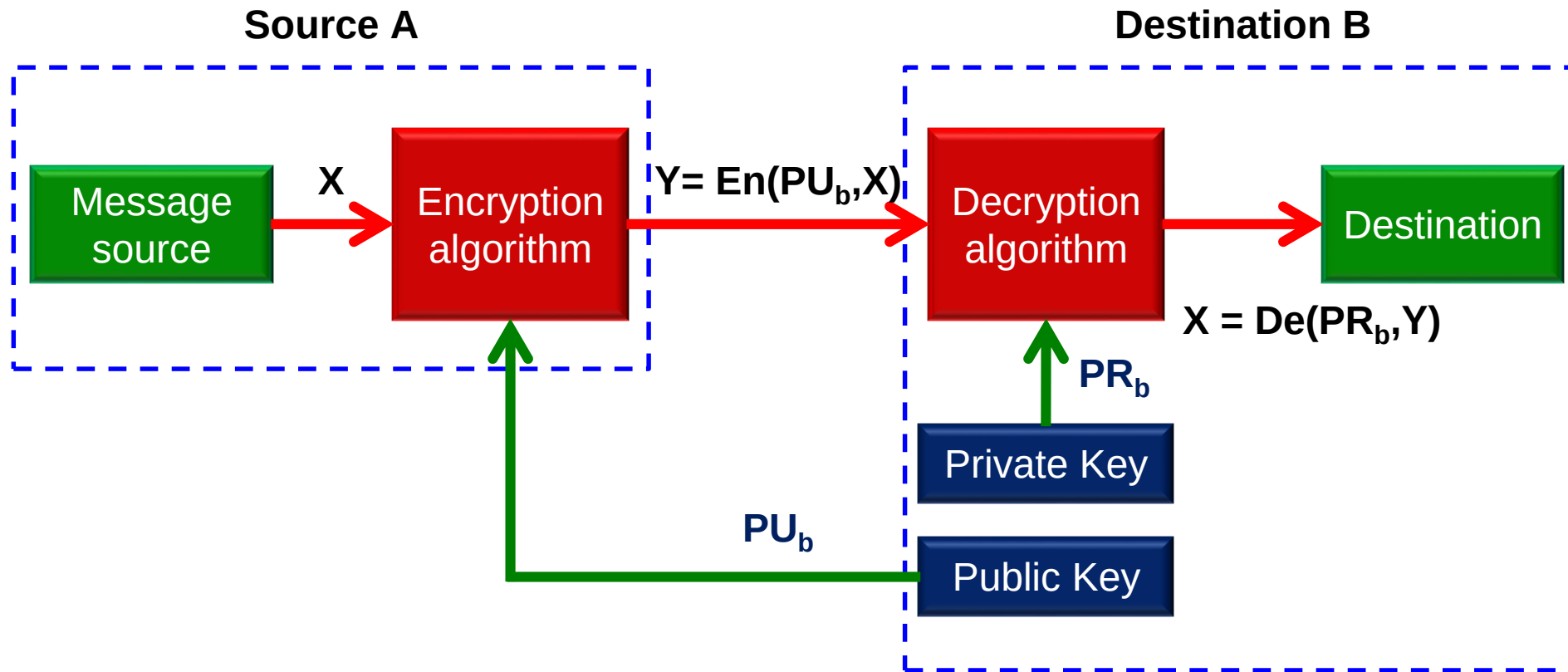


ASYMMETRIC

Asymmetric cryptography, using key pairs for each of its users, has n as the number of key pairs needed. In a situation with 1000 users, that would mean 1000 key pairs.

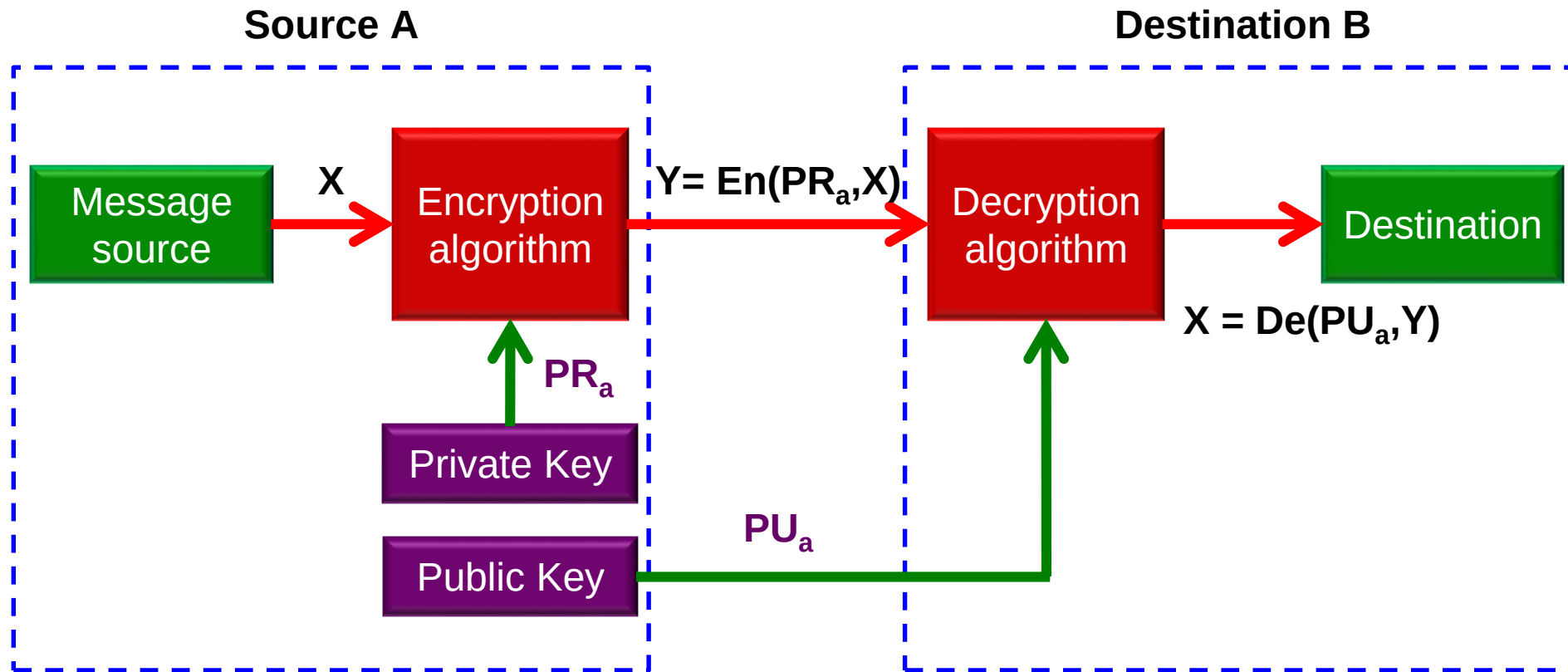
PUBLIC KEY CRYPTOGRAPHY (CONT.)

- **Encryption with Public Key** → Provides Confidentiality (Secrecy)



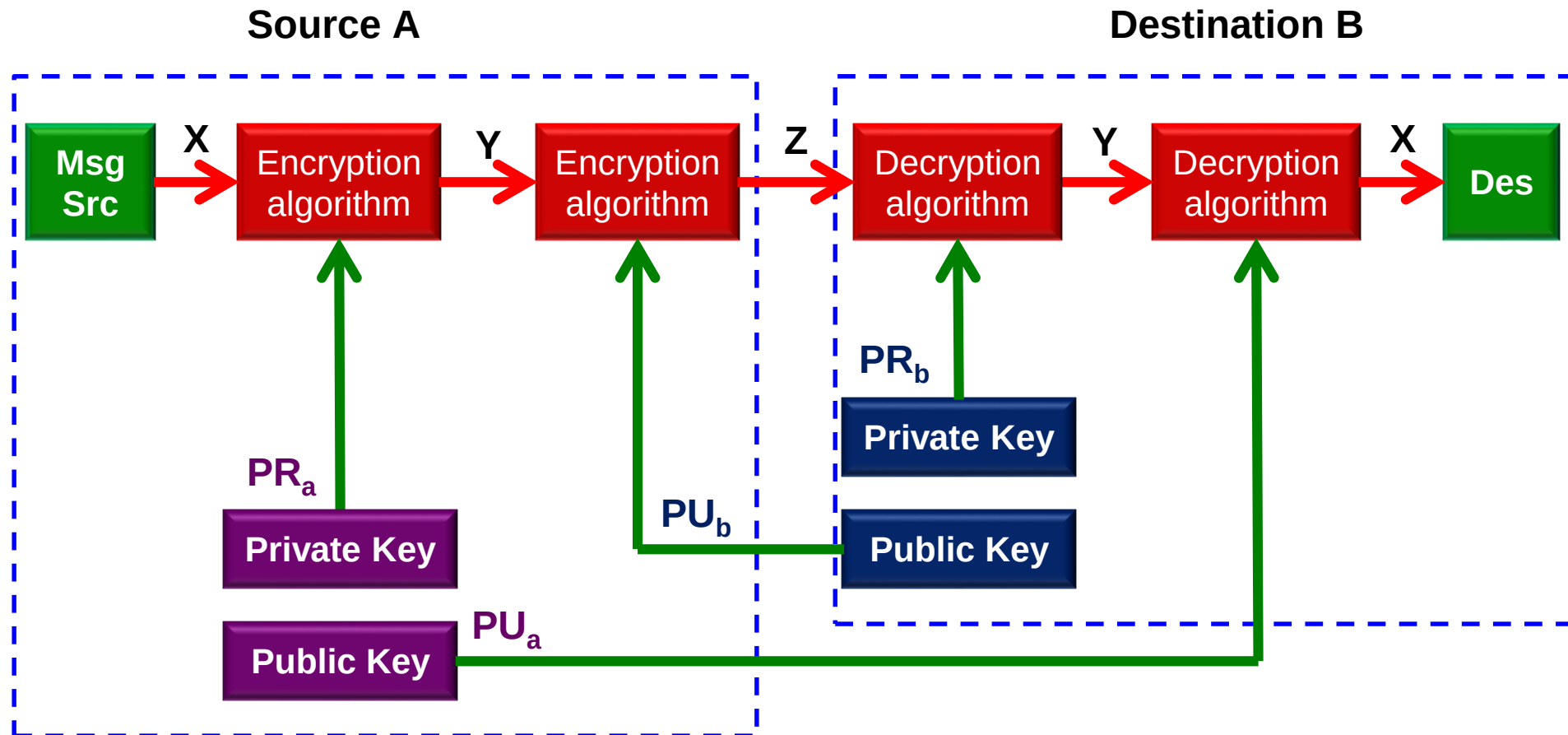
PUBLIC KEY CRYPTOGRAPHY (CONT.)

- **Encryption with Private Key** → Provides **Integrity** (Authentication)



PUBLIC KEY CRYPTOGRAPHY (CONT.)

- Encryption with Private Key & Public Key → Provides Authentication and Secrecy

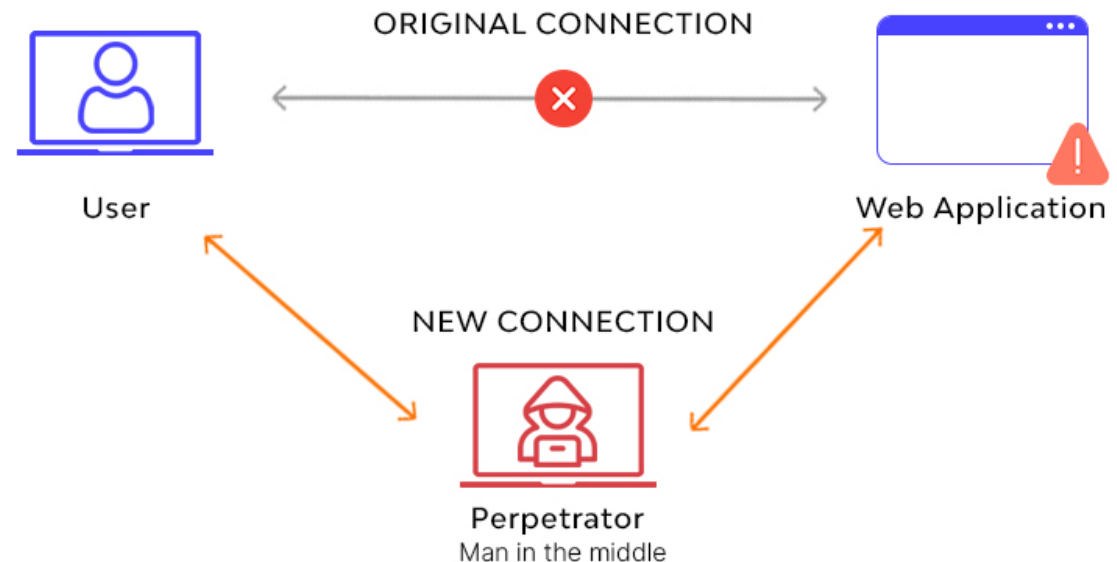


PUBLIC KEY CRYPTOGRAPHY (CONT.)

- The **advantages** of public-key encryption include:
 - It reduces the risks and time associated with transmitting the decryption key to the recipient—issues often encountered with symmetric-key encryption.
 - It eliminates key distribution problems between the sender and the recipient.
- The **disadvantages** of this method include:
 - The public key can be misused by unauthorized parties.
 - Computational processes are slower and involve greater algorithmic complexity compared to symmetric-key encryption.
 - It requires larger key sizes than symmetric-key encryption.

PUBLIC KEY CRYPTOGRAPHY (CONT.)

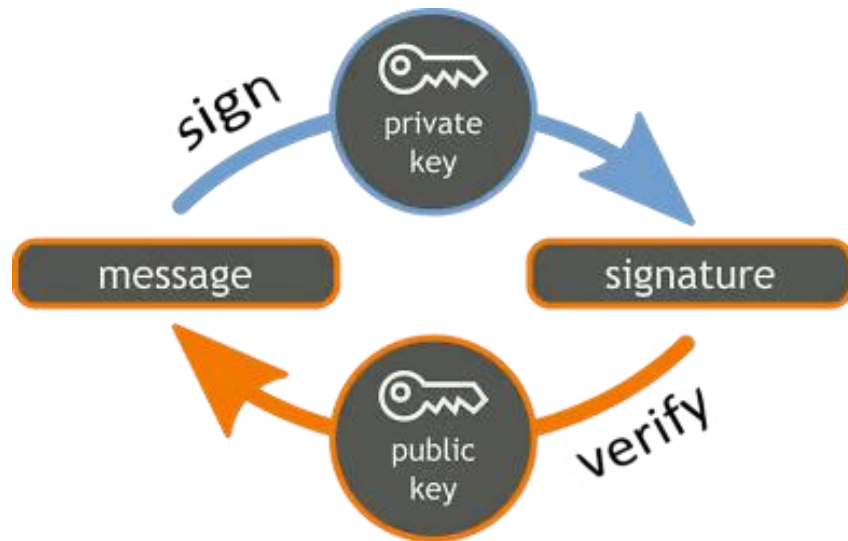
- In reality, hybrid cryptography does not fully resolve the key distribution problem, as a *new issue arises*: how can we be certain that both parties have truly obtained each other's public keys?
- The primary issue with public-key encryption is its vulnerability to **man-in-the-middle attacks**.
 - Since Alice and Bob are exchanging public keys over an insecure channel, a third party (Eve) could potentially be eavesdropping.
 - During this exchange, Eve might send her own public key to both Alice and Bob; if this happens, it is possible that *Alice and Bob could end up transmitting data through Eve*.



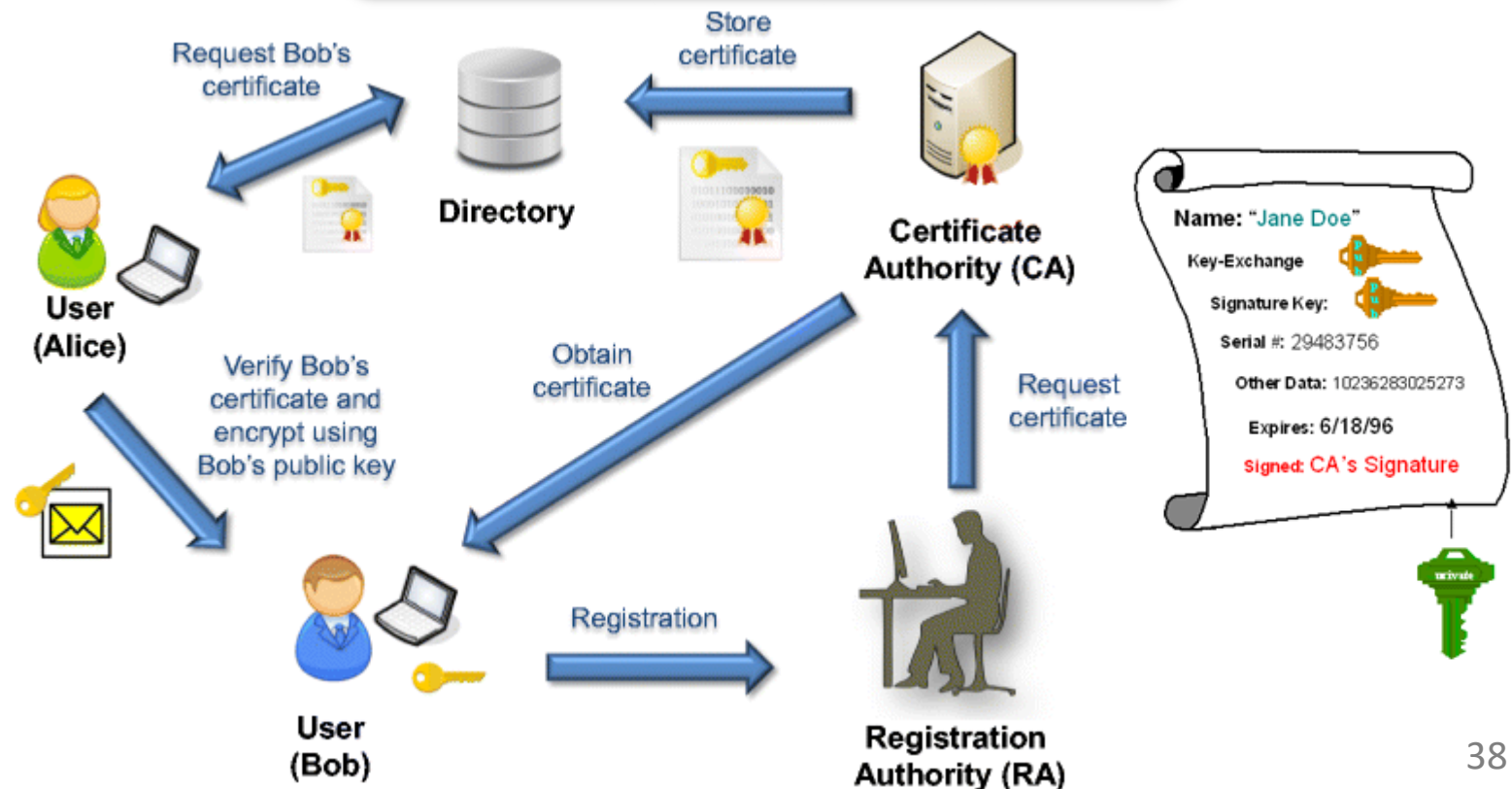
PUBLIC KEY CRYPTOGRAPHY (CONT.)

This problem can be solved by using ...

Digital Signatures



Public Key Infrastructure (PKI)



Brute Force Attack on RSA

Why it is difficult, what attackers actually try, and when it can be easier to break.

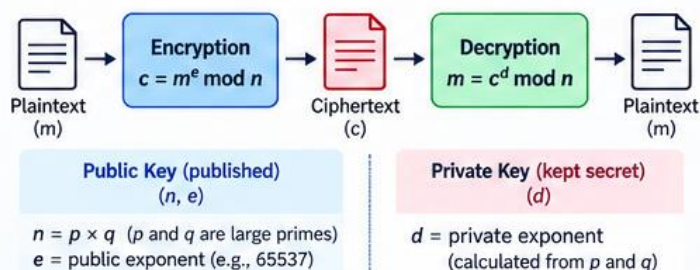


Key Takeaway

RSA is not broken by simply trying all possible keys. Its security relies on the difficulty of factoring a large integer and the correct implementation of the system.

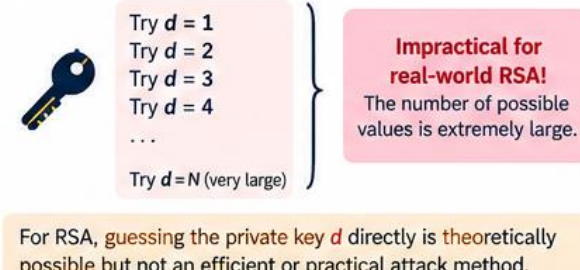
1 How RSA Works (Briefly)

RSA uses a pair of keys: a public key for encryption and a private key for decryption.



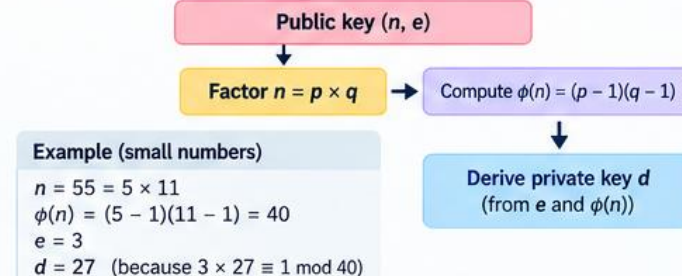
2 What is a Brute Force Attack?

Brute force means trying every possible value until the correct one is found.



3 The Main Challenge: Integer Factorization

The real way to break RSA is to factor the modulus n into its prime factors p and q .



4 Example: Small RSA (Easy to Break)

Using small prime numbers, factorization and key recovery are very simple.

$$p = 5, q = 11 \rightarrow n = 55$$
$$\phi(n) = 40, e = 3 \rightarrow d = 27$$

Encryption

$$m = 7$$
$$e = 7^3 \bmod 55$$
$$= 343 \bmod 55$$
$$= 13$$

Decryption

$$m = 13^{27} \bmod 55$$
$$= 7$$

(recovered successfully)

5 Real-World RSA Key Sizes

Real RSA uses very large prime numbers. Factoring such large numbers is computationally infeasible with classical computers.

RSA Key Size (modulus n)	Approximate Size (in decimal digits)	Status / Security Level
RSA-512	155 digits	Insecure (deprecated)
RSA-768	232 digits	Broken (factored)
RSA-1024	309 digits	Not secure for new systems
RSA-2048	617 digits	Common minimum (~112-bit security)
RSA-3072	925 digits	Higher security (~128-bit security)
RSA-4096	1,234 digits	Stronger but higher performance cost

6 Comparison with AES Brute Force

AES security is based on the size of the secret key, so brute force means trying all possible keys.

AES-128 (Symmetric Key)



Key space: 2^{128}
 $\approx 3.4 \times 10^{38}$ possibilities

- Brute force tries every key.
- No practical way to try all keys with current technology.
- Considered secure for most modern applications.

RSA-2048 (Asymmetric Key)



Not 2^{2048} security!

- The attack focuses on factoring n , not trying 2^{2048} keys.
- The best known classical algorithm is the General Number Field Sieve (GNFS).
- Estimated security level is around 112 bits, not 2048 bits.

7 Why RSA Brute Force / Factoring Is Hard



Very large numbers
 p and q are typically 1,024 bits each (for RSA-2048).



Best classical algorithms
GNFS is the most efficient known algorithm, but still requires enormous computational resources and time.



Not simple trial division
Attackers do not try every number one by one.



Exponential computational cost
Even with large-scale distributed computing, factoring RSA-2048 is currently infeasible.

8 Factors That Can Make RSA Easier to Break

In practice, RSA can be broken more easily if there are weaknesses in key generation, implementation, or protocol design.



Weak Random Number Generation
Predictable primes or low entropy.



Shared Prime Factors
Multiple keys using the same prime factor (can be found with GCD).



Improper Padding / Protocol Design
Vulnerable to attacks (e.g., padding oracle attacks).



Side-Channel Attacks
Leaking information via timing, power consumption, or fault analysis.



Small or Poorly Chosen e
Can lead to specific attacks in some cases.



Implementation Bugs
Incorrect checks or vulnerabilities in libraries/devices.

9 What About Quantum Computers?

A sufficiently powerful quantum computer can run **Shor's Algorithm**, which can factor large integers efficiently.



Today, such quantum computers do not exist, but they are a future risk. This is one reason why Post-Quantum Cryptography (PQC) is being developed.

ADDITIONAL RESOURCES

- https://simple.wikipedia.org/wiki/RSA_algorithm
- <https://legacy.cryptool.org/en/cto/rsa-step-by-step>
- <https://legacy.cryptool.org/en/cto/rsa-visual>
- <https://www.geeksforgeeks.org/computer-networks/public-key-encryption>
- <https://www.geeksforgeeks.org/computer-networks/rsa-algorithm-cryptography>